A Classical Reasoning Agent for Symbolic Entailment

Artificial Intelligence Assignment Report

Shivam Nitin Solanki
Roll No.: P26CS0009
Artificial Intelligence
Indian Institute of Technology Jodhpur

31 October 2026

Abstract

This report describes a Python reasoning agent that evaluates a knowledge base and a query using classical inference. It supports symbolic first-order representations, a restricted ground-rule representation, and Boolean structural causal models. Inference combines forward chaining, unification, resolution, DPLL satisfiability search, and verified finite-model search. The latest logged public-data run contains 4,029 queries from RuleTaker, CounterBench, and FOLIO: 3,682 predictions are correct, giving 91.39% binary accuracy with 26.530062 seconds of recorded inference time. These aggregate results include training splits. Unsupported or resource-limited cases are reported separately, although their binary output is a `False` fallback.

1 Task and Scope

For a knowledge base $\mathcal{K}$ and a closed query q, the formal reasoning task is to determine whether $\mathcal{K} \models q$. The evaluation considered here begins with a supplied internal representation. Public-dataset adapters normalize supported source formats before reasoning. The final prediction is produced by the classical engine; reference labels are used only for scoring.

The implementation is contained in `reasoning_agent.ipynb`. Each output record has an identifier and a Boolean `flag`. Additional in-memory diagnostics include a resolution status, a reason, a trace, and prediction time. This report covers the representation, reasoning algorithms, public experiments, and illustrative traces required by the assignment.

2 Knowledge Representation

The formal engine represents atoms as a predicate name, an argument tuple, and a negation flag. Terms are constants, variables, or function terms; logical connectives and quantifiers form a recursive syntax tree. For example,

$$\mathsf{Young}(Bob), \qquad \forall x\,(\mathsf{Young}(x) \rightarrow \mathsf{Smart}(x)).$$

Input can use formula strings, structured atoms and rules, or clauses. Variables in queries must be explicitly quantified. Symbol names are case sensitive and quantifier scope is validated.

The ground-rule adapter stores normalized sentences as propositional atoms and single-premise definite-Horn rules as directed implications. The causal adapter uses Boolean structural equations, declared root variables, observations, and interventions. Routing is based on the input schema rather than dataset names or reference answers.

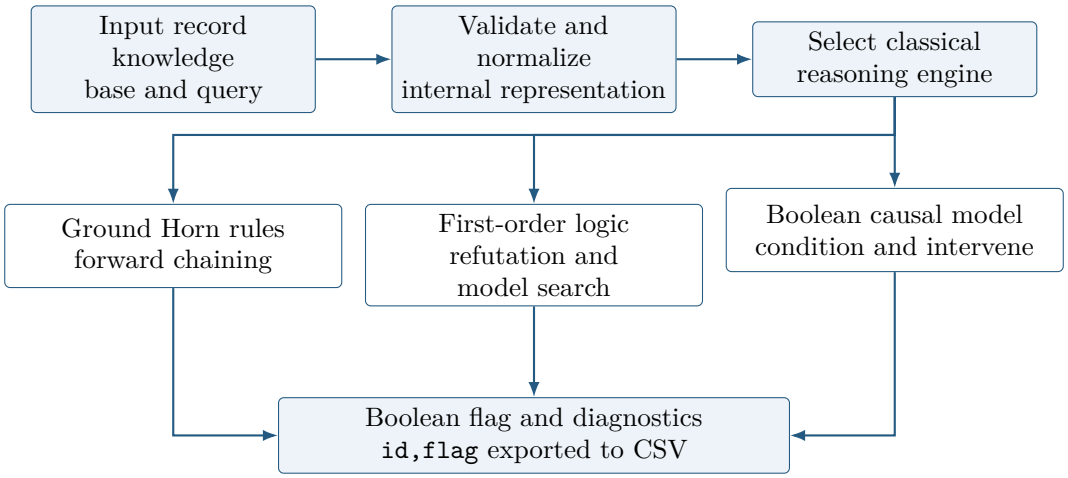


Figure 1: Implementation overview. Schema-specific normalization is followed by classical inference. Labels are outside the prediction path.

3 Classical Reasoning Algorithms

3.1 First-Order Refutation and Satisfiability

The main entailment test uses the refutation equivalence

$$\mathcal{K} \models q \quad \Longleftrightarrow \quad \mathcal{K} \wedge \neg q \text{ is unsatisfiable.} \tag{1}$$

The engine also tests $\mathcal{K} \wedge q$ for diagnostics. A countermodel of $\mathcal{K} \wedge \neg q$ establishes non-entailment even when $\neg q$ is not entailed. Logical uncertainty and an implementation timeout are distinct.

After parsing and scope validation, each probe follows this sequence:

1. Standardize bound variables apart, Skolemize existential quantifiers with their dependencies, and build polarity-aware definitional CNF using auxiliary predicates.
2. Attempt a bounded unit-resolution refutation using unification with an occurs check.
3. Where function-free grounding is feasible, ground the clauses and use DPLL with unit propagation, pure literals, branching, and lazy equality consistency checks.
4. If grounding is unavailable, search for finite interpretations on domains of sizes one to three and independently verify candidate models against the original parsed formulas.
5. If no verified model is found, add equality axioms as needed and apply first-order resolution with unification and factoring.

Failure to find a small finite model does not prove unsatisfiability. Limits on time, grounding, or clause growth can leave a probe unresolved; the bounded implementation is not claimed to decide arbitrary first-order theories.

Table 1: Diagnostic outcomes when both satisfiability probes complete.

$\mathcal{K} \wedge \neg q$	$\mathcal{K} \wedge q$	Status	Binary flag
Unsatisfiable	Satisfiable	`true`	`True`
Satisfiable	Unsatisfiable	`false`	`False`
Satisfiable	Satisfiable	`unknown`	`False`
Unsatisfiable	Unsatisfiable	`both`	`True`

The `both` case follows classical entailment for an inconsistent knowledge base. The binary decision is resolved when the first probe completes; a diagnostic `unknown` can therefore have `resolved=True`.

A failed parse or unresolved first probe instead has `resolved=False` and emits `False` as an explicit fallback.

3.2 Ground Horn Forward Chaining

For supported ground facts F_0 and rules $a \to b$, the adapter computes

$$F_{t+1} = F_t \cup \{b \mid (a \to b) \in R,\ a \in F_t\}$$

until no new atom is added. Query membership in the resulting closure determines entailment within this restricted language. Provenance records the antecedent used for each derived atom. This adapter supports single-premise ground templates. Quantified or conjunctive rules must be supplied in the supported formal representation and use the formal engine.

3.3 Boolean Causal Reasoning

The causal engine enumerates admissible Boolean root assignments, retains factual worlds consistent with observations, replaces intervened equations, and evaluates compatible post-intervention worlds with the same selected roots. It resolves a query when all admissible conditioned worlds agree. Acyclic equations are propagated; cyclic remainders are checked for fixed points. This is support-based reasoning rather than probability estimation. Disagreement, inconsistent evidence, or unsupported model features are recorded in diagnostics. Causal models are limited to 16 variables.

4 Public-Dataset Experiments

4.1 Protocol and Data

The latest supplied run uses `jise/ruletaker`, `CounterBench/CounterBench`, and `yale-nlp/FOLIO` from Hugging Face [1, 2, 3]. The evaluated RuleTaker repository contains 1,625 records across train, test, and OOD test; this subset is distinguished from the entire original RuleTaker benchmark [4]. CounterBench contributes 1,200 question records from its alpha and backdoor files; the loader excludes 1,200 query-free model-blueprint support records. FOLIO contributes 1,204 records across train and validation, using its supplied FOL annotations [5].

Reference labels are read after prediction. FOLIO's 393 *Uncertain* references map to `False` for binary entailment. Notebook metadata records Python 3.11.15. The per-query reasoning budget is 2.0 seconds. Recorded inference time includes routing, parsing, and reasoning, but excludes dataset downloads and CSV writing. Hardware details were not recorded; timing is a single-run measurement.

4.2 Accuracy, Coverage, and Cost

With N examples and C matching predictions,

$$\text{Accuracy} = 100\,C/N, \qquad \text{Coverage} = 100\,N_{\text{resolved}}/N.$$

Coverage indicates that the reasoning path established its decision, not that it matched the reference. Accuracy includes fallback predictions. The combined accuracy is micro-averaged over all examples.

Table 2: Latest logged public run. Times are in seconds.

Dataset	Queries	Correct	Accuracy	Resolved	Time
RuleTaker	1,625	1,600	98.46%	1,625	0.569105
CounterBench	1,200	1,135	94.58%	1,180	0.350887
FOLIO	1,204	947	78.65%	977	25.610071
Combined	4,029	3,682	91.39%	3,782	26.530062

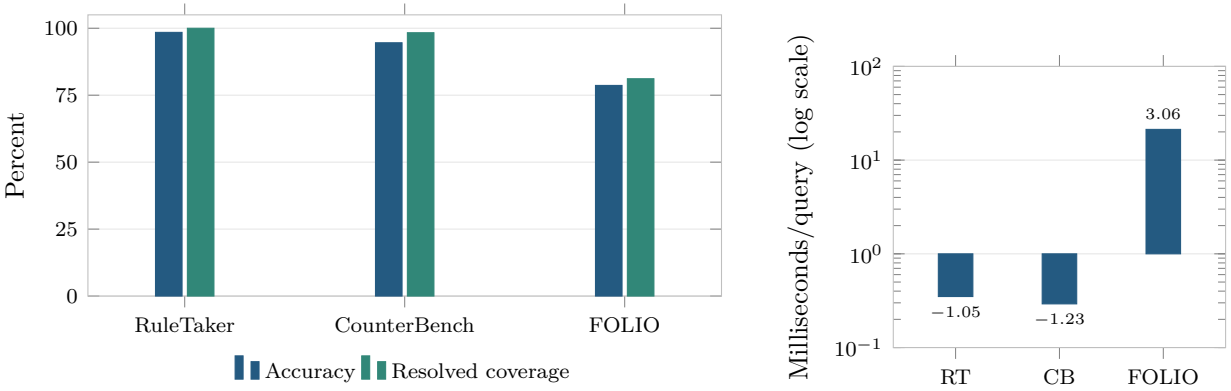


Figure 2: Accuracy and resolved coverage (left); mean recorded prediction time per query (right). RT and CB abbreviate RuleTaker and CounterBench. The time axis is logarithmic.

Table 3: Split-level accuracy from the same run.

Dataset	Split / partition	Correct / queries	Accuracy
RuleTaker	Train	983 / 1,000	98.30%
	Test	244 / 250	97.60%
	OOD test	373 / 375	99.47%
CounterBench	Alpha (train)	955 / 1,000	95.50%
	Backdoor (train)	180 / 200	90.00%
FOLIO	Train	783 / 1,001	78.22%
	Validation	164 / 203	80.79%

The aggregate includes training data and is not an estimate of private-test accuracy. FOLIO accounts for approximately 96.53% of recorded prediction time and has the lowest coverage. Its inference path dominates the recorded runtime; stage-specific parsing and solving times were not measured separately.

5 Examples of Reasoning Traces

These illustrative inputs were executed against the submitted implementation. They explain behavior and are separate from the public-data accuracy calculation.

5.1 Multi-Step Entailment

The knowledge base is

$$\begin{aligned}\mathcal{K} = \{&\mathsf{Young}(Bob),\ \mathsf{Kind}(Bob),\\ &\forall x\,(\mathsf{Young}(x) \to \mathsf{Smart}(x)),\\ &\forall x\,((\mathsf{Smart}(x) \wedge \mathsf{Kind}(x)) \to \mathsf{Successful}(x))\}.\end{aligned}$$

For $q = \mathsf{Successful}(Bob)$, the first rule derives $\mathsf{Smart}(Bob)$; the second then derives the query. Adding $\neg q$ causes a contradiction. The actual recorded backend summary is:

```
KB + not(query):  contradiction proved by
proof-only unit resolution (8 clauses).
KB + query:  satisfiability established by
finite DPLL with lazy equality (8 clauses).
```

The output is `flag=True`, `status=true`, and `resolved=True`. The derivation above is an explanatory reconstruction; the notebook records backend outcomes rather than a complete exported clause-by-clause proof.

5.2 Verified Countermodel and Non-Entailment

Consider

$$\mathcal{K} = \{\forall x\,\exists y\,R(x,y)\}, \qquad q = \exists y\,\forall x\,R(x,y).$$

A verified interpretation is

$$D = \{0,1\}, \qquad R = \{(0,0),(1,1)\}.$$

Each element relates to some element, satisfying $\mathcal{K}$, but no single element is related to by every element, so q is false in this interpretation. The engine finds this countermodel for $\mathcal{K} \wedge \neg q$ and also establishes satisfiability of $\mathcal{K} \wedge q$. Its trace reports:

```
KB + not(query):  satisfiability established by
independently verified finite interpretation (2 clauses).
KB + query:  satisfiability established by
independently verified finite interpretation (2 clauses).
```

The output is `flag=False`, `status=unknown`, and `resolved=True`: established binary non-entailment, not a timeout fallback.

5.3 Observation Followed by Intervention

Let $Y := U$ and $Z := Y$ be Boolean structural equations with root U. Factual observation $Y = \text{True}$ selects $U = \text{True}$. Under $\text{do}(Y = \text{False})$, the equation for Y is replaced, while the selected root remains fixed. The counterfactual outcome is $Z = \text{False}$:

```
Roots {'U': True}; do {'Y': False};
query=False; evidence timing=factual.
```

For query Z, the output is `flag=False`, `status=false`, and `resolved=True`.

6 Limitations and Error Analysis

The run contains 247 unresolved cases and 347 incorrect predictions. These sets overlap: a fallback can match the reference, while a resolved prediction can disagree with it. Table 4 groups unresolved diagnostics, not all incorrect predictions. Syntax and scope rejection together account for 218 cases. These include malformed source formulas and scope mismatches; the counts alone do not establish that every rejection is a parser bug.

Table 4: Unresolved diagnostics in the latest public run.

Diagnostic category	Count
Formula syntax	200
Variable scope	18
Conflicting counterfactual evidence	10
Undefined intervention variable	5
Conflicting factual evidence	5
Input representation	5
Inference time limit	4
Total	247

Finite-model search is restricted to small domains, resolution is bounded, and the ground-language adapter accepts a restricted grammar. Larger causal models and unsupported evidence/prior structures can also remain unresolved. These restrictions limit coverage; no completeness claim is made for arbitrary representations or first-order theories.

Practical improvements are to document accepted syntax and scope conventions, distinguish malformed annotations from valid unsupported forms, retain diagnostics alongside predictions, and evaluate tuning on separate public validation data. The binary fallback policy should remain visible when reporting accuracy.

7 Reproducibility and Submission

The evaluated repository revisions are recorded below. They identify the supplied run; `HF_REVISION=None` otherwise resolves the current revision.

Repository	Recorded revision
`jise/ruletaker`	`b6e36448e186cde51052e978da0465d84da06f31`
`CounterBench/CounterBench`	`c6225dfa00b7a89ccfeae67797b9a6d80e48e44c`
`yale-nlp/FOLIO`	`4c1f14bf39ca6a6de242bd8f75ef313b4b436818`

For the public configuration, use the three repositories in consecutive slots, leave slots four and five as `None`, and use `HF_SPLIT=None`, `LABEL_COLUMN='auto'`, and the 2.0-second timeout. Exact source pinning can be set per dataset slot. Timing and timeout-dependent predictions can vary across machines.

For a TA's private file, set slot one to its path and the other slots to `None`, restart the kernel, and run all cells. Labels are optional for prediction. The CSV contains exactly `id,flag`; unique supplied IDs are preserved, missing IDs are generated, and cross-partition collisions are namespaced. The supplied public output contains 4,029 unique IDs. The separate `README.txt` provides installation, input examples, column mapping, and output locations.

8 Conclusion

The implementation combines complementary classical procedures behind a common Boolean interface. The latest public run achieves 91.39% binary accuracy and 93.87% resolved coverage. Its strongest results are on the evaluated ground-rule and causal subsets; FOLIO exposes the main syntax, coverage, and computational limitations. These results describe the observed public run and do not establish private-test performance.

References

[1] jise, *ruletaker*, Hugging Face dataset repository. https://huggingface.co/datasets/jise/ruletaker. Accessed 31 October 2026.

[2] CounterBench, *CounterBench*, Hugging Face dataset repository. https://huggingface.co/datasets/CounterBench/CounterBench. Accessed 31 October 2026.

[3] Yale NLP, *FOLIO*, Hugging Face dataset repository. https://huggingface.co/datasets/yale-nlp/FOLIO. Accessed 31 October 2026.

[4] P. Clark, O. Tafjord, and K. Richardson, *Transformers as Soft Reasoners over Language*, in Proc. IJCAI, 2020, pp. 3882–3890. https://doi.org/10.24963/ijcai.2020/537.

[5] S. Han et al., *FOLIO: Natural Language Reasoning with First-Order Logic*, in Proc. EMNLP, 2024, pp. 22017–22031. https://doi.org/10.18653/v1/2024.emnlp-main.1229.